
(DRAFT) Graph Neural Networks for Predicting Flaviviral Genomic Capping Enzyme Inhibition (PCBA 588689)

William Bruns
Stanford CGOE Student
adde.animulis@gmail.com

Abstract

Flaviviruses (Yellow Fever, Dengue, West Nile Virus, Japanese Encephalitis, etc) cause great morbidity and mortality annually worldwide but as yet have no approved antivirals. The common Flaviviral Genomic Capping Enzyme essential to viral replication might be a promising antiviral target. (Southern Research Specialized Biocontainment Screening Center (2011)) Graph neural networks have showed promise in molecular property prediction aiding in screening large numbers of molecules for promising candidates before expensive laboratory testing (Wong et al. (2024), Liu et al. (2023), Stokes et al. (2020), Wu et al. (2018), and Ramsundar et al. (2015)). Here, we evaluate the Open Graph Benchmark (OGB) (Hu et al. (2021)) baseline Graph Isomorphism Network (GIN) (Xu et al. (2019)) and Graph Convolutional Network (GCN) (Kipf and Welling (2017)) on predicting the ability of small molecules to inhibit the Flaviviral Genomic Capping Enzyme as well as suggest an alternate hyperparameter regime resulting in a model more than 93% smaller than the smallest competing baseline and 99% smaller than the baseline GIN w/ virtual node (best AP of baselines) with superior performance.

Split	Nodes	Edges	Graphs	Average Nodes per Graph	Average Edges per Graph	Positive Class Graphs	Positive %
Overall	8621717	18586230	338853	25.44	54.85	1013	0.2989%
Train	6839247	14686128	271082	25.23	54.18	758	0.2796%
Validation	893445	1951302	33885	26.37	57.59	128	0.3777%
Test	889025	1948800	33886	26.24	57.51	127	0.3747%

Table 1: PCBA 588689 Dataset statistics. The graphs are split by Bemis-Murcko molecular scaffold split, sorted by descending scaffold cardinality (most common scaffolds in train, then validation, then test) with ties broken by random ordering. The test set is all unique scaffolds. "Positive class graphs" in this binary classification problem refer to the Active Molecules, i.e. those that would inhibit the Flaviviral Genomic Capping Enzyme. The scaffold splitting ensures that the Active Molecules in the validation and test splits are not structurally similar to any in the training set.

1 Data

We start with the public PubChem BioAssay 588689 dataset (Southern Research Specialized Biocontainment Screening Center (2011)), a CSV of Simplified Molecular Input Line Entry System (SMILES) (Weininger (1988)) strings with "Active", "Inactive", and "Inconclusive" status for inhibiting the Flaviviral Genomic Capping Enzyme.¹ We collapse "Inconclusive" into "Inactive" to give a binary classification problem of predicting the confirmed "Active" molecules.

Before splitting, we have 338,853 molecular graphs total, with 1,013 "Active" molecules (positive class graphs), and 337,840 "Inactive"/"Inconclusive" molecules (negative class graphs). The dataset statistics for each split are described in detail in Table 1.

To avoid the network simply matching similar molecules between the train and test sets separated by random splitting and enforce generalization (since some molecules differ only by a few atoms with the overall scaffold/backbone identical), we use a Bemis-Murcko Molecular Scaffold Split (Bemis and Murcko (1996)) to obtain train, validation, and test splits of the data, as done by the Pande lab MoleculeNet team (Wu et al. (2018)) for the molhiv dataset, using the code they contributed to DeepChem v2.8.0 (https://deepchem.readthedocs.io/en/2.8.0/api_reference/splitters.html#scaffoldsplitter)². We use 80% train, 10% validation, 10% test split fractions as is common. Molecular scaffolds are sorted by descending scaffold cardinality (most common scaffolds in train, then validation, then test) with ties broken by random ordering. The test set is all unique scaffolds.

We then use the OGB loader (Hu et al. (2021)) to convert the SMILES strings for each molecule into Pytorch Geometric (Fey and Lenssen (2019)) compatible graphs.³

Dataset preparation code is available to the public at <https://github.com/willy-b/tiny-GIN-for-WNV/>.

¹We note that we have not filtered the molecules present in the original high-throughput screening by the PCBA 588689 authors against any filter catalogs, like PAINS (Baell and Holloway (2010), Capuzzi et al. (2017)) or BRENK (Brenk et al. (2008)). The popular RDKit comes with a list of such catalogs,

<https://github.com/rdkit/rdkit/blob/144c1f29f10146ad390eb492cf31e5eb1136b858/Code/GraphMol/FilterCatalog/README#L1>, and this approach is used to filter predicted active molecules found by deep learning guided drug discovery in Wong et al. (2024). The use of such catalogs appears not unanimously supported with some noting that e.g. use of the PAINS catalog would have flagged multiple FDA approved drugs as undesirable during early screening Capuzzi et al. (2017).

²Per Wu et al. (2018): "Scaffold splitting splits the samples based on their two-dimensional structural frameworks, as implemented in RDKit. Since scaffold splitting attempts to separate structurally different molecules into different subsets, it offers a greater challenge for learning algorithms than the random split. ... MoleculeNet contributes the code for these splitting methods into DeepChem. Users of the library can use these splits on new datasets with short library calls."

³`ogb.utils.mol.smiles2graph` and `ogb.io.save_dataset.DatasetSaver`, and `ogb.graphpropred.PygGraphPropPredDataset` from `ogb 1.3.6`

Model	Parameter Count	GNN Layers	Hidden Dim	Has Virtual Node	Pooling Type	MLP after pooling	Normalization	Weight Decay
OGB Team GIN	1,885,506	5	300	False	Mean	False	Batch	0
OGB Team GIN w/ virtual node	3,336,606	5	300	True	Mean	False	Batch	0
OGB Team GCN	528,001	5	300	False	Mean	False	Batch	0
OGB Team GCN w/ virtual node	1,979,101	5	300	True	Mean	False	Batch	0
Tiny GIN	32,449	2	64	False	Sum	True	GraphNorm	1e-6

Table 2: GNN Models trained from scratch on the PCBA 588689 train split and evaluated on PCBA 588689 test split. "GNN layers" refers to number of GCN/GIN blocks used to compute the node embedding and number of hops from each node for which information is aggregated in computing that nodes embedding. "Pooling Type" refers to the aggregation used to transform the node embeddings for a graph into the single graph embedding of hidden dimension for that graph (e.g. by sum, mean, max pooling). "MLP after pooling" refers to whether after pooling the node embeddings to obtain a graph-level embedding there is a linear transformation (if False) or a nonlinear transformation (linear transformation, nonlinearity, linear transformation; if True) to the final 1-dimensional logit used for binary classification of the graph. The Tiny GIN used batch size 128 instead of 32 (small effect to use 32 instead, and only on ROCAUC, no stat. sig. effect on AP). Other than batch size and use of GraphNorm, the Tiny GIN hyperparameters are identical to those used by the author in the ogbg-molhiv competition (see <https://github.com/willy-b/tiny-GIN-for-ogbg-molhiv>) (on predicting HIV antiviral activity). All parameter counts computed using 'sum(p.numel() for p in model.parameters())'.

2 Methods and Models

Hyperparameters/configurations and parameter counts for all models are described in Table 2.

2.1 Baseline OGB Team GIN/GCN Models

We use the OGB team baseline models for molecular property prediction (used in Hu et al. (2021)) from their repo⁴, comprising GIN (Xu et al. (2019)) and GCN (Kipf and Welling (2017)) models with and without virtual node (Gilmer et al. (2017), Li et al. (2017)),

replacing their construction of their PygGraphPropPredDataset⁵,

with the setup of the PygGraphPropPredDataset as in our repo⁶,

running the dataset prep script from our repo⁷.

See the OGB baseline Test Evaluation Notebooks for the model configuration used at:

- OGB Team GIN baseline (1,885,506 parameters)⁸
- OGB Team GIN w/ virtual node baseline (3,336,606 parameters)⁹
- OGB Team GCN baseline (528,001 parameters)¹⁰
- OGB Team GCN w/ virtual node baseline (1,979,101 parameters)¹¹

⁴<https://github.com/snap-stanford/ogb/tree/861511315ad4b4082d10662d081c3cae14cfc3ea/examples/graphproppred/mol>

⁵https://github.com/snap-stanford/ogb/blob/861511315ad4b4082d10662d081c3cae14cfc3ea/examples/graphproppred/mol/main_pyg.py

⁶https://github.com/willy-b/tiny-GIN-for-WNV/blob/4c7098f834c4eb4c6591ae194ec7662fab4df582/main_gin.py

⁷https://github.com/willy-b/tiny-GIN-for-WNV/blob/4c7098f834c4eb4c6591ae194ec7662fab4df582/prep_pcba/prep.py

⁸<https://colab.research.google.com/drive/18fwSQ4CzsDOZmcqUBIJc1S3yvtvxxUwGt>

⁹<https://colab.research.google.com/drive/1fCrvEImy8FxB06pKd91NLMlnYAYJAY3y>

¹⁰<https://colab.research.google.com/drive/1QPyJK8Q7b6xxQaV8LIcAw1p-xKNHD-w>

¹¹https://colab.research.google.com/drive/1R16OFplusMD5pBbmMQh0WZFHip6_XbIb

Note that the Test Evaluation Notebooks linked above also link back to the relevant validation notebooks.

We also update the OGB Team `main_pyg.py` to report Average Precision (in addition to ROCAUC).

2.2 Custom Tiny GIN model

All of our code is available at <https://github.com/willy-b/tiny-GIN-for-WNV> and the full PyTorch Model description with all dimensions and code generating it is in the Appendix: Tiny GIN Model Details.

For atomic features, we use the 9 dimensional representation provided by `ogb.graphproppred.mol_encoder.AtomEncoder`.¹²

We use PyTorch Geometric (Fey and Lenssen (2019)) (version 2.5.3) for training and inference of the custom Tiny GIN model.

Specifically, we use `torch_geometric.nn.GINConv` and `torch_geometric.nn.models.MLP` layers, `torch_geometric.nn.global_add_pool` for pooling, `torch_geometric.nn.norm.GraphNorm` for normalization and `torch_geometric.loader.DataLoader` for dataloading.

We also use the base PyTorch (Paszke et al. (2019)) heavily, for `torch.nn.Module`, `torch.nn.Linear`, `torch.nn.functional.relu`, and `torch.nn.functional.dropout` (50% dropout used when applicable).

For optimization we use `torch.optim.Adam` (Kingma and Ba (2014)) with a learning rate of 0.001 and weight decay of 1e-6 (Hinton and Van Camp (1993)).

We used just two GNN blocks¹³ (instead of OGB team's five) with a hidden dimension of 64 (instead of OGB team's 300) and added a nonlinearity after the node embeddings were pooled into a graph embedding (to avoid assuming the raw pooled node embeddings would be linearly separable) based on hyperparameter and architecture exploration done for the ogbg-molhiv challenge earlier¹⁴ (for entirely different dataset but was found in the validation set to generalize to this one as well, so reused it with only an increase in batch size to compensate for lower active molecule fraction and addition of GraphNorm).

The test set evaluation notebook has been made available¹⁵, which shows the configuration and also links to the earlier validation set notebook.

2.3 Evaluation

Though this is not an OGB dataset, we use OGB evaluators (Hu et al. (2021)): `ogb.graphproppred.PygGraphPropPredDataset` is used to represent the dataset once loaded and the existing OGB ROCAUC evaluator for molecular property binary classification from OGB for ogbg-molhiv provided by `ogb.graphproppred.Evaluator` (just calls `sklearn.metrics.roc_auc_score` underneath but like to be consistent with ogbg-molhiv, an existing, popular antiviral prediction problem for HIV).

`sklearn.metrics.roc_auc_score` and `sklearn.metrics.average_precision_score` are used (OGB tools use these implementations underneath so numbers are consistent whether reported through the Evaluator

¹²Atomic number, Chirality, Degree, Formal Charge, Number of hydrogens, Hybridization, Number of radical electrons, Aromatic, In ring

¹³See discussion in, for example Xu et al. (2018): "Yet, such aggregation schemes sometimes lead to surprises. For example, it has been observed that the best performance with one of the state-of-the-art models, Graph Convolutional Networks (GCN), is achieved with a 2-layer model. Deeper versions of the model that, in principle, have access to more information, perform worse Kipf and Welling (2017). A similar degradation of learning for computer vision problems is resolved by residual connections He et al. (2015) that greatly aid the training of deep models. But, even with residual connections, GCNs with more layers do not perform as well as the 2-layer GCN on many datasets[.]" Later they put it as: "Large radii [more GNN blocks] may lead to too much averaging, while small radii may lead to instabilities or insufficient information aggregation." We tried jumping knowledge networks (Xu et al. (2018)'s suggested solution to allow more GNN blocks while mitigating downsides) for the ogbg-molhiv competition and did not find an advantage at cost of greater parameter count so did not use it here.

¹⁴see that work at <https://github.com/willy-b/tiny-GIN-for-ogbg-molhiv>

¹⁵https://colab.research.google.com/drive/1-xdSI71kmMhC76ha7S5kbhkrI27_h6a0

or via sklearn directly and both approaches are used, e.g. using sklearn directly for ROCAUC and PRC curves).

Model	Parameter Count	Valid ROCAUC % (mean +/- std) (95% CI)	Valid AP % (mean +/- std) (95% CI)
OGB Team GIN	1,885,506	94.8 +/- 0.3% (94.6 to 95.0%)	14.9 +/- 1.2% (14.2 to 15.7%)
OGB Team GIN w/ virtual node	3,336,606	94.7 +/- 0.4% (94.4 to 94.9%)	17.4 +/- 2.4% (15.9 to 18.9%)
OGB Team GCN	528,001	93.4 +/- 0.4% (93.1 to 93.6%)	13.3 +/- 1.3% (12.5 to 14.1%)
OGB Team GCN w/ virtual node	1,979,101	94.6 +/- 0.4% (94.4 to 94.8%)	14.1 +/- 1.4% (13.2 to 15.0%)
Tiny GIN	32,449	94.1 +/- 0.2% (94.0 to 94.2%)	19.0 +/- 1.7% (17.9 to 20.0%)

Table 3: Results for the models evaluated on the PCBA 588689 **validation set** (overestimates used for checking hyperparameters **before doing real evaluation on test set**). N=10 separate runs with different random weight initialization and training data permutation for all models. **For each of N=10 runs, Best of M (after M training epochs for each random initialization) validation is reported for ROCAUC with M=50 for Tiny GIN and M=100 for OGB models per their runner, so validation ROCAUC is expected to be overoptimistic vs test (and OGB more over-optimistic than Tiny GIN).** Best of M epochs model by ROCAUC has its AP reported for each of N=10 training from scratch runs, from N=10 separate runs the average and variation are reported (approach used by OGB team training and evaluation script.)

Model	Parameter Count	Test ROCAUC % (mean +/- std) (95% CI)	Test AP % (mean +/- std) (95% CI)
OGB Team GIN	1,885,506	91.1 +/- 1.0% (90.5 to 91.7%)	12.1 +/- 1.0% (11.6 to 12.6%)
OGB Team GIN w/ virtual node	3,336,606	91.8 +/- 1.3% (91.0 to 92.6%)	13.3 +/- 0.7% (12.9 to 13.7%)
OGB Team GCN	528,001	92.3 +/- 0.5% (92.0 to 92.6%)	11.2 +/- 0.8% (10.7 to 11.7%)
OGB Team GCN w/ virtual node	1,979,101	91.8 +/- 0.8% (91.3 to 92.3%)	12.9 +/- 2.1% (11.6 to 14.2%)
Tiny GIN	32,449	93.8 +/- 0.4% (93.5 to 94.1%)	13.9 +/- 0.8% (13.5 to 14.4%)

Table 4: Results for the models evaluated on the PCBA 588689 **test set**. N=10 separate runs with different random weight initialization and training data permutation for all models.

3 Results

Validation set performance for all models is given in Table 3.

Test set performance for all models is given in Table 4.

A representative receiver operating characteristic curve for the best model, Tiny GIN, for its random seed 1 (out of 10 seeds) is given in Figure 1, and a precision recall curve (PRC) for the same model is given in Figure 2.

Predicting if molecules inhibit Flavivirus Genome Capping Enzyme
32449 parameter 2-hop GIN with GraphNorm and hidden dimension 64

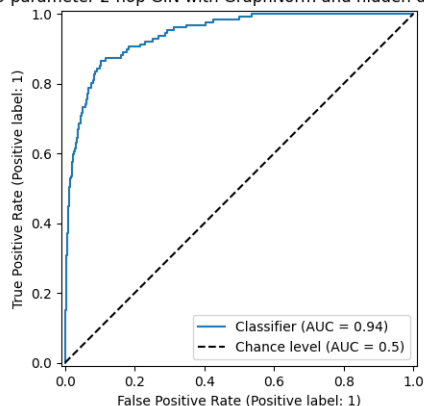


Figure 1: Receiver Operating Characteristic for best model (Tiny GIN) on predicting ability of molecule to inhibit the Flaviviral Genomic Capping Enzyme (held-out test set performance). Seed 1 is shown here (1 of 10 random weight initializations trained from scratch with randomized training data ordering and evaluated). See notebook for all 10 seeds.

Predicting if molecules inhibit Flavivirus Genome Capping Enzyme
32449 parameter 2-hop GIN with GraphNorm and hidden dimension 64

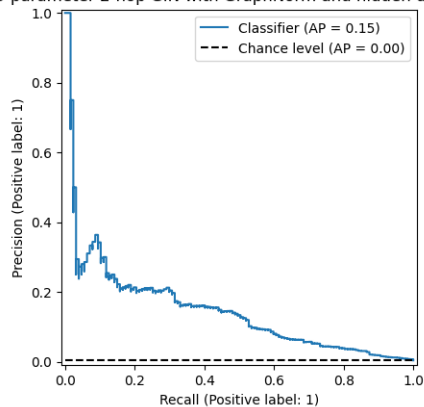


Figure 2: Precision Recall Curve for best model (Tiny GIN) on predicting ability of molecule to inhibit the Flaviviral Genomic Capping Enzyme (held-out test set performance). Seed 1 is shown here (1 of 10 random weight initializations trained from scratch with randomized training data ordering and evaluated). See notebook for all 10 seeds.

References

- Jonathan B Baell and Georgina A Holloway. 2010. New substructure filters for removal of pan assay interference compounds (pains) from screening libraries and for their exclusion in bioassays. *Journal of medicinal chemistry*, 53(7):2719–2740.
- Guy W. Bemis and Mark A. Murcko. 1996. The properties of known drugs. 1. molecular frameworks. *Journal of Medicinal Chemistry*, 39(15):2887–2893. PMID: 8709122.
- Ruth Brenk, Alessandro Schipani, Daniel James, Agata Krasowski, Ian Hugh Gilbert, Julie Frearson, and Paul Graham Wyatt. 2008. Lessons learnt from assembling screening libraries for drug discovery for neglected diseases. *ChemMedChem: Chemistry Enabling Drug Discovery*, 3(3):435–444.
- Stephen J Capuzzi, Eugene N Muratov, and Alexander Tropsha. 2017. Phantom pains: Problems with the utility of alerts for pan-assay interference compounds. *Journal of chemical information and modeling*, 57(3):417–427.
- Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with pytorch geometric.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural message passing for quantum chemistry.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep residual learning for image recognition.
- Geoffrey E Hinton and Drew Van Camp. 1993. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the sixth annual conference on Computational learning theory*, pages 5–13.
- Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2021. Open graph benchmark: Datasets for machine learning on graphs.
- Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks.
- Junying Li, Deng Cai, and Xiaofei He. 2017. Learning graph-level representation for drug discovery.
- Gary Liu, Denise B Catacutan, Khushi Rathod, Kyle Swanson, Wengong Jin, Jody C Mohammed, Anush Chiappino-Pepe, Saad A Syed, Meghan Fragis, Kenneth Rachwalski, et al. 2023. Deep learning-guided discovery of an antibiotic targeting *acinetobacter baumannii*. *Nature Chemical Biology*, 19(11):1342–1350.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.
- Bharath Ramsundar, Steven Kearnes, Patrick Riley, Dale Webster, David Konerding, and Vijay Pande. 2015. Massively multitask networks for drug discovery.
- Southern Research Specialized Biocontainment Screening Center. 2011. Pubchem bioassay record for aid 588689, primary and confirmatory screening for flavivirus genomic capping enzyme inhibition.
- Jonathan M Stokes, Kevin Yang, Kyle Swanson, Wengong Jin, Andres Cubillos-Ruiz, Nina M Donghia, Craig R MacNair, Shawn French, Lindsey A Carfrae, Zohar Bloom-Ackermann, et al. 2020. A deep learning approach to antibiotic discovery. *Cell*, 180(4):688–702.

- David Weininger. 1988. Smiles, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36.
- Felix Wong, Erica J Zheng, Jacqueline A Valeri, Nina M Donghia, Melis N Anahtar, Satotaka Omori, Alicia Li, Andres Cubillos-Ruiz, Aarti Krishnan, Wengong Jin, et al. 2024. Discovery of a structural class of antibiotics with explainable deep learning. *Nature*, 626(7997):177–185.
- Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. 2018. Moleculenet: A benchmark for molecular machine learning.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks?
- Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation learning on graphs with jumping knowledge networks. In *International conference on machine learning*, pages 5453–5462. PMLR.

Acknowledgments and Disclosure of Funding

This work was self-funded. The author is excited that the Tiny GIN configuration introduced here can be trained from scratch on a standard laptop CPU in a relatively short period of time or be trained on the CPU free tier of Google Colab and other cloud platforms to enable others to more easily use GNNs for these types of problems in a personal research or classroom setting where even paying for a few hours of powerful GPU time per person per experiment is prohibitive (OGB models were trained on A100s in the Cloud; Tiny GIN was essentially free).

Thanks to Stanford XCS224W for teaching the author about Graph Neural Networks and introducing them to the Open Graph Benchmark (OGB) challenges and tools, and PyTorch Geometric;

to the UC San Diego Data Science MicroMasters program for giving the author a foundation in designing experiments and performing analysis;

and thanks to the OGB team for allowing the author to participate in the ogbg-molhiv OGB benchmark challenge earlier.

Finally, the author thanks his artificial life partner Ying Li who is a continual source of inspiration for work and projects.

A Appendix / supplemental material

A.1 Tiny GIN Model Details

The Tiny GIN PyTorch Model Description is as follows:

```
GINGraphPropertyModel(
  (node_encoder): AtomEncoder(
    (atom_embedding_list): ModuleList(
      (0): Embedding(119, 64)
      (1): Embedding(5, 64)
      (2-3): 2 x Embedding(12, 64)
      (4): Embedding(10, 64)
      (5-6): 2 x Embedding(6, 64)
      (7-8): 2 x Embedding(2, 64)
    )
  )
  (convs): ModuleList(
    # note that each of the OGB team baseline model GIN layers
    # also have a MLP inside
    (0-1): 2 x GINConv(nn=MLP(64, 64, 64))
  )
)
```

```

)
(bns): ModuleList(
  (0): GraphNorm(64)
)
(linear_hidden): Linear(in_features=64, out_features=64, bias=True)
# nonlinearity between these layers, AFTER node embedding pooling unlike OGB GNNs
(linear_out): Linear(in_features=64, out_features=1, bias=True)
)

```

The source code to generate our Tiny GIN is as follows:

(for entire program see our repository¹⁶)

```

config = {
    'device': 'cpu',
    'dataset_id': 'ogbg-pcba-aid-588689',
    'num_layers': 2,
    'hidden_dim': 64,
    'dropout': 0.5,
    'learning_rate': 0.001,
    'epochs': 50,
    'batch_size': 128,
    'weight_decay': 1e-6,
    'grad_norm_clip_max_value': None,
    'use_graph_norm': True,
    'split_type': 'scaffold'
}

class GINGraphPropertyModel(torch.nn.Module):
    def __init__(self, hidden_dim, output_dim, num_layers, dropout_p, return_node_embed=False):
        super(GINGraphPropertyModel, self).__init__()
        # fields used for computing node embedding
        self.node_encoder = AtomEncoder(hidden_dim)
        self.return_node_embed = return_node_embed
        self.convs = torch.nn.ModuleList(
            [torch_geometric.nn.conv.GINConv(MLP([hidden_dim, hidden_dim, hidden_dim])) for idx in range(0, num_layers)]
        )
        if config['use_graph_norm']:
            self.bns = torch.nn.ModuleList(
                [GraphNorm(hidden_dim) for idx in range(0, num_layers - 1)],
            )
        else:
            self.bns = torch.nn.ModuleList(
                [torch.nn.BatchNorm1d(hidden_dim) for idx in range(0, num_layers - 1)]
            )
        self.dropout_p = dropout_p
        # end fields used for computing node embedding
        # fields for graph embedding
        self.pool = global_add_pool
        self.linear_hidden = torch.nn.Linear(hidden_dim, hidden_dim)
        self.linear_out = torch.nn.Linear(hidden_dim, output_dim)
        # end fields for graph embedding
    def reset_parameters(self):
        for conv in self.convs:
            conv.reset_parameters()
        for bn in self.bns:
            bn.reset_parameters()
        self.linear_hidden.reset_parameters()
        self.linear_out.reset_parameters()
    def forward(self, x, edge_index, batch):
        # compute node embedding
        x = self.node_encoder(x)
        for idx in range(0, len(self.convs)):
            x = self.convs[idx](x, edge_index)
            if idx < len(self.convs) - 1:
                if config['use_graph_norm']:
                    x = self.bns[idx](x, batch)
                else:
                    x = self.bns[idx](x)
                    x = torch.nn.functional.relu(x)
            x = torch.nn.functional.dropout(x, self.dropout_p, training=self.training)
        # note x is raw logits, NOT softmax'd
        # end computation of node embedding
        if self.return_node_embed == True:
            return x
        # convert node embedding to a graph level embedding using pooling
        x = self.pool(x, batch)
        x = torch.nn.functional.dropout(x, self.dropout_p, training=self.training)
        # transform the graph embedding to the output dimension
        # MLP after graph embed ensures we are not requiring the raw pooled node embeddings to be linearly separable
        x = self.linear_hidden(x)
        x = torch.nn.functional.relu(x)
        x = torch.nn.functional.dropout(x, self.dropout_p, training=self.training)
        out = self.linear_out(x)

```

¹⁶<https://github.com/willy-b/tiny-GIN-for-WNV>

return out